Google Developer Groups

On Campus • IIT Mandi

KRACKHACK

THEMES

Prize Distribution

10K INR Per Domain

100$ Requesty Bounty open to all Domains

5K Bounty on AI/ML Track by Gold Sponsors

RACKHACK

01 Mobile App Security Analyzer

Cybersecurity : Problem Explanation:

Mobile applications frequently contain security vulnerabilities such as hardcoded API keys, weak cryptography, insecure configurations, improper exported components, and excessive permissions. Many developers lack accessible tools to quickly analyze APK files for such risks.

This problem asks you to build a static analysis security tool that evaluates Android APK files and reports structured security findings.

What You Are Expected to Build

A system that:

- Accepts an Android APK file (iOS IPA support optional bonus)
- Performs static analysis (no runtime or dynamic instrumentation required)
- Detects common vulnerabilities
- Maps findings to OWASP Mobile Top 10
- Generates a structured security report with severity levels
- Suggests remediation guidance
- The focus is detection accuracy and report quality - not just scanning output

Evaluation Metrics:

A GitHub repository link is expected as your submission, containing a README file with clear instructions on how to run your code. (mandatory)

1.**Detection Coverage:** How effectively your tool identifies important mobile security vulnerabilities. The broader and more accurate the vulnerability detection, the better your coverage.

2**. False Positive Control:** Your tool should avoid reporting issues that are not actually security risks. High-quality tools minimize unnecessary or incorrect alerts.

3. Correct OWASP Mapping: Detected vulnerabilities must be correctly categorized according to the OWASP Mobile Top 10 framework. Proper classification shows understanding of security standards.

4. Clarity and Usefulness of Generated Report: The final report should be easy to read, well-structured, and actionable. It should clearly explain the issue, its severity, and recommended remediation steps.

5. Code Quality and Usability: Your implementation should follow clean coding practices, be well-organized, and easy to run. A simple and intuitive interface (CLI or web) will be valued.

RACKHACK

What We Expect:

- Public GitHub repository
- Working tool (CLI or Web interface)
- Clear README with setup instructions
- Sample report output
- Short demo video explaining workflow

For any queries, contact - Harsh(+91 95188 30309)

02 PayStream: Real-Time Payroll Streaming

Problem Statement:

Track : Blockchain:

Traditional payroll systems are rigid, operating on monthly or bi-weekly cycles that don't align with the real-time needs of the modern workforce. While blockchain-based streaming (like Sablier or Superfluid) exists, high and volatile gas fees on networks like Ethereum make it impractical for businesses to pay hundreds of employees simultaneously.

PayStream aims to disrupt this by leveraging the HeLa Network. By utilizing HeLa's stablecoin-native gas architecture (HLUSD), businesses can achieve predictable, low-cost transaction fees. This makes "Salary Streaming" where an employee earns their pay second-by-second commercially viable for the first time.

Resources for Beginners:

- HeLa Network Documentation: HeLa Labs Official Docs
- Understanding Payroll Streaming: Research "Money Streaming" protocols and ERC-20 payment flows.
- Smart Contract Development: * Solidity Docs
    - OpenZeppelin Contracts
- Web3 Frontend: Ethers.js or Viem

Evaluation Metrics:

A GitHub repository link is expected as your submission, containing a README file with clear instructions on how to run your code. (mandatory).

- Streaming Efficiency: How accurately the smart contract calculates per-second value transfer without rounding errors.
- Gas Optimization: Leveraging HeLa's HLUSD for gas to ensure low-cost operations.
- Security: Proper access control (only HR/Admin can start streams) and withdrawal safety.
- UI/UX: A clean dashboard for HR to manage payroll and a "claim" interface for employees.
- Compliance Features: Implementation of basic logic for tax withholding or automated deductions.

RACKHACK

What We Expect:

- Blockchain Layer: Deployment on the HeLa Testnet.
- Smart Contract Development: Solidity-based streaming logic using HLUSD as the primary asset.
- HR Dashboard: A management portal to "Start/Pause/Cancel" streams and view treasury balances.
- Employee Portal: A simplified view for workers to track their earnings and "Withdraw" accrued funds to their HeLa wallet.
- Tax Module: A basic logic gate that redirects a percentage (e.g., 10%) of the stream to a designated "Tax Vault."

Additional Suggestions:

- Bonus (On/Off Ramps): Integrate a mock or real ramp service to show how an employee converts HLUSD to local currency.
- Yield Integration: While the salary is sitting in the streaming vault, can it earn a small yield for the employer or employee?
- Scheduled Bonuses: Ability to trigger "one-time" streaming spikes for performance bonuses.

Contact & Support:

HeLa Developers: @HeLa_Developers (Telegram)

Discord: https://discord.gg/vjFsTBypdr

03 The AEGIS Protocol

Web D: Problem Explanation:

Campus systems often operate in silos - grievance systems, academic tracking, internship listings, announcements, and community tools are scattered across platforms.
This problem challenges you to build a unified digital ecosystem for campus life that integrates governance, academics, opportunities, and student interaction.

What You Are Expected to Build

A web-based platform that includes: Mandatory Core Components

1. Role-based authentication (Student, Faculty, Authority, Admin)
2. Grievance management system with status tracking
3. Academic resource and course tracking module
4. Internship/research opportunity portal

You may also implement additional optional modules such as ride-sharing, lost & found, discussion forum, club management, etc. The emphasis is on system design, role separation, usability, and integration — not just feature count.

Evaluation Metrics:

A GitHub repository link is expected as your submission, containing a README file with clear instructions on how to run your code. (mandatory)

Criterion	Weight	Description
Architectural Integrity	20%	Code cleanliness, database schema design, security protocols, scalability considerations, and documentation quality
Pillar Completeness	40%	Functionality of implemented pillars (core and bonus). Quality matters more than quantity - exceptional execution of fewer features can outweigh incomplete implementation of all features. However, strong core pillar coverage is highly valued.
The Seeker's Experience	25%	UI/UX quality, intuitiveness, responsiveness, accessibility, and overall smoothness of the user journey across different roles
Oracle's Innovation	15%	Creative solutions, bonus features implemented, thoughtful additions beyond requirements, and technical cleverness

What To Submit

- GitHub repository
- Live deployment link (if possible)
- Demo video (5–7 minutes)
- Technical documentation (architecture + setup guide)

Refer To Detailed Doc -

Web Dev - PS KrackHack

For any queries, contact - Vansh(+91 78400 52725),

Vinamra Garg(+91 9350555053)

RACKHACK

04 Offroad Semantic Scene Segmentation

AI/ML : Problem Explanation:

Autonomous ground vehicles rely on semantic segmentation to understand terrain. Real-world data collection for off-road environments is expensive and limited. This challenge provides synthetic desert environment data generated via digital twins. You must train a segmentation model and evaluate its ability to generalize to unseen terrain.

What You Are Expected to Build

- Train a semantic segmentation model using the provided dataset
- Evaluate performance on unseen test images
- Optimize IoU score
- Analyze model failure cases
- Maintain proper train/validation/test separation

You must use only the provided dataset.

Evaluation Metrics:

A GitHub repository link is expected as your submission, containing a README file with clear instructions on how to run your code. (mandatory)

IoU Score (Primary Metric)

Indicates how accurately your model segments the image compared to the actual ground truth. Higher IoU reflects better segmentation performance.

Model Robustness

Your model should perform well not only on training data but also on unseen test images.

Documentation Clarity

Your report should clearly explain your training approach, results, and improvements made.

Proper ML Workflow Discipline

Maintain proper separation between training and testing data and follow standard model evaluation practices.

RACKHACK

Links and Detailed Doc:

AI/ML Detailed PS - Sponsored by Duality.AI

For any queries, contact - Akshat Mittal (+91 87662 26219)

RACKHACK

05 The Sovereign AI Challenge

Gen AI: Problem Explanation:

Most AI projects rely on external APIs. This challenge focuses on building task-specific, proprietary Small Language Models (SLMs) that run independently. The goal is to identify a focused, high-value problem and train a specialized model that solves it efficiently.

What You Are Expected to Build

- Define a narrow AI use case
- Generate or synthesize training data
- Train a specialized model using Smolify
- Deploy the model as part of your application
- Ensure inference does not depend on external APIs

The emphasis is on specialization and efficiency.

Evaluation Metrics:

A GitHub repository link is expected as your submission, containing a README file with clear instructions on how to run your code and a google drive link containing the weights for your Deep Learning Model (Mandatory). Deployment of your code will earn you bonus points.

Real-world Utility

Your solution should address a practical problem and demonstrate meaningful impact in a real use-case scenario.

Model Efficiency

The trained model should be lightweight and optimized for performance without unnecessary computational overhead.

Integration into Application

The model should be properly integrated into your application and actively used in the solution workflow.

Technical Clarity

Your implementation and documentation should clearly explain how the model works and how it contributes to solving the problem.

RACKHACK

What We Expect:

- Working prototype powered by your trained model
- Documentation explaining the problem and solution
- Clear explanation of why a specialized model is appropriate
- Repository with reproducibility steps

Links and Detailed Doc:

Start building at:

https://smolify.ai

Documentation:

https://smolify.ai/docs/get-started

For any queries, contact - Abhinav (+91 92058 02993), Vipresh (+91 79 7357 4307)

100$ - Sponsor Bounty

Requestly - Creative Use Prize

Open to all tracks

This bounty rewards the most effective and advanced use of the Requestly API Client during development.

What Qualifies

- Complex API request construction
- Authentication handling
- Pre/post-response scripting
- Endpoint validation
- Debugging workflows

What to Submit for Bounty

- Screen recording showing Requestly in use
- Short write-up explaining how it improved your workflow Proof of usage is mandatory for eligibility

Detailed Doc and Links

Requestly - Challenge

FOLLOW US:

Instagram

Linkedin

X

Website